CS383 · SOFTWARE ENGINEERING

Lecture 4 Agile & Scrum

Process Models (II) · ملخص الميد

محارب

محتوى الملف

01 ليش طلع الـ Agile؟

02 Agile methodology

03 Extreme Programming (XP)

04 User stories

05 Plan-driven vs Agile

06 Scrum

07 حسابات الـ Sprint

08 ملخص النماذج الأربعة

09 كيف يجي في الميد؟

10 أخطاء شائعة

11 اختبر نفسك

01 ليش طلع الـ Agile؟

في الواقع الشركات تخلط بين النماذج العامة حسب حاجة المشروع. واختيار النموذج يتأثر بـ: العقود والأنظمة، خبرة الفريق بالنموذج، والنماذج المستخدمة في مشاريع مرتبطة.

اليوم البرامج تنبني بسرعة والمتطلبات تتغير بسرعة. فالمواصفات والتصميم والتنفيذ صارت متداخلة، والنظام ينبني على دورات (**Iterations**).

المشكلة

الـ **Plan-driven** في هذي الحالة يصير عليه حِمل زايد (**Overhead**) من الإجراءات والوثائق.

02 Agile methodology

- التركيز على الكود، مو على التصميم والوثائق.
- إصدارات صغيرة للعميل كل 2 إلى 3 أسابيع.
- مناسب للمنتجات الصغيرة والمتوسطة.
- يحتاج عميل ملتزم ومشارك في التطوير.

المبادئ الخمسة

Principle	معناه
Customer involvement	العميل مشارك طول فترة التطوير. يعطي المتطلبات ويرتّب أولوياتها ويقيّم كل دورة.
Incremental delivery	البرنامج ينبني على دفعات، والعميل يحدد متطلبات كل دفعة.
People not process	نعتمد على مهارات الفريق ونخليهم يشتغلون بطريقتهم.
Embrace change	نتوقع أن المتطلبات تتغير، ونصمم النظام عشان يتحمل.
Maintain simplicity	نركّز على البساطة في البرنامج وفي طريقة الشغل.

عيوب الـ Agile

- صعب تخلّي العميل مهتم طول الوقت.
- بعض أعضاء الفريق ما يناسبهم الشغل المكثّف.
- العضو الجديد يحتاج وثائق عشان يفهم النظام.
- تغيير الأولويات صعب إذا كثروا الـ stakeholders.
- البساطة نفسها تحتاج شغل زيادة.

03 Extreme Programming (XP)

طريقة Agile متطرفة في التطوير على دورات. دورتها:

```
Select user stories  →  Break down stories to tasks  →  Plan release
```

```
Develop / integrate / test  →  Release software  →  Evaluate system
```

الممارسات الثمان

#	Practice	معناها
1	**Incremental planning**	المتطلبات تنكتب على story cards وتترتب بالأولوية.
2	**Small releases**	إصدارات صغيرة ومتكررة كل 2 إلى 3 أسابيع.
3	**Simple design**	تصميم يكفي الحاجة الحالية بس.
4	**Test-first development**	اختبار آلي لكل وظيفة تنضاف.
5	**Refactoring**	تحسين الكود باستمرار.
6	**Pair programming**	مبرمجين يراجعون كود بعض ويرتبونه.
7	**Continuous integration**	كل مهمة تخلص تندمج في النظام، بشرط تنجح كل الاختبارات.
8	**On-site customer**	العميل متاح طول الوقت.

جاء في الميد

سؤالين MCQ: الممارسة اللي تحسّن الكود باستمرار هي **Refactoring**. ومن مبادئ XP الأساسية: **Continuous feedback**.

ملاحظتين من السلايد

- التطوير على دفعات يخرّب بنية الكود مع الوقت. الـ XP يحلها بالـ **Refactoring** المستمر.
- كثرة الاختبارات الآلية ما تعني أن الاختبار شامل. المبرمج ممكن يختصر ويكتب اختبارات ناقصة.

User stories 04

صاحب المشروع والمطورين يكتبون الـ **User stories**. كل story فيها ثلاثة أشياء:

الجزء	معناه
Who	دور المستخدم: عميل، موظف، مدير نظام.
What	الهدف: وش الوظيفة المطلوبة.
Why	السبب: ليش يحتاجها.

As a user, I want a cascade action warning before updating or deleting a record, because I want to avoid dependency problems.

وممكن ينحط لكل story عدد نقاط تقديري (**Story points**).

Plan-driven vs Agile 05

في الـ **Plan-driven** تخلص المتطلبات أول وتنكتب في وثيقة، وبعدها التصميم والتنفيذ. في الـ **Agile**

المتطلبات والتصميم والتنفيذ يمشون مع بعض.

الحالة	Plan-driven	Agile
Very detailed specification and design before implementation	✓	
Deliver software to customers and get rapid feedback		✓
Large systems development	✓	
Systems require a lot of analysis before implementation	✓	
Long-lifetime system is required	✓	
Development may require up-to-date technologies for support		✓
Development team is distributed (outsourced)	✓	
Development team has strong rapid development skills		✓
The system is subject to regulation (external entity)	✓	

Scrum 06

إدارة المشاريع العادية تناسب الـ Plan-driven. الـ Agile يحتاج طريقة ثانية، وأشهرها **Scrum**. يمشي على

ثلاث مراحل:

#	المرحلة	وش يصير
1	**Initial phase**	نحدد الأهداف العامة ونصمم الـ architecture.
2	**Sprint cycles**	كل دورة من 2 إلى 4 أسابيع، وتطلّع جزء جديد من النظام.
3	**Closure phase**	نقفل المشروع ونكمل الوثائق.

Sprint planning

لازم يصير قبل ما تبدأ الدورة.

المدخلات (Inputs)	المخرجات (Outputs)
Team capacity	Sprint goal
Product backlog	Sprint backlog
Business conditions	
Current product	
Technology	

07 حسابات الـ Sprint

هذا السؤال الحسابي في المحاضرة. فيه ثلاثة قوانين تمشي ورا بعض.

1 متوسط النقاط المنجزة — SPRINT VELOCITY

متوسط الـ story points المنجزة في آخر 3 إلى 5 دورات.

```
Sprint Velocity = sum of completed story points ÷ number of sprints
```

2 قدرة الفريق على التركيز — FOCUS FACTOR

```
Focus Factor = Sprint Velocity ÷ (people × hours per day)
```

المدى المثالي من `0.6` إلى `0.8` .

3 ساعات الشغل الفعلية — EFFECTIVE TEAM CAPACITY

```
Capacity = Focus Factor × (people × hours per day × working days)
```

الأسبوع فيه 5 أيام شغل. نشيل نهاية الأسبوع.

مثال السلايد

فريق من 5 أشخاص، 8 ساعات باليوم، ومتوسط النقاط 30. الدورة الجاية 3 أسابيع.

```
Focus Factor = 30 ÷ (5 × 8) = 30 ÷ 40 = 0.75
```

```
Working days = 3 × 5 = 15
```

```
Capacity = 0.75 × (5 × 8 × 15) = 0.75 × 600 = 450 hours
```

معلومة

الـ **Team capacity** تتحدد بدقة بعد الدورة الثالثة أو الرابعة، لأن وقتها يكون عندنا بيانات فعلية عن أداء الفريق.

08 ملخص النماذج الأربعة

	Waterfall	Incremental	Reuse-oriented	Agile
Requirements volatility	Low	Medium	Low	High
Project size	Large	Medium	Any	Small
Customer involvement	Low	Medium	Low	High
Technical risk	Low	High	Medium	High
Release schedule	Long	Medium	Short	Very short

احفظه

هذا الجدول مفتاح سؤال اختيار النموذج، وجاء في الميدين بـ 4 درجات.

09 كيف يجي في الميد؟

السؤال الحسابي (الميد الأول · 4 درجات)

Team of 6 people; workday = 8h. Last 4 sprints completed story points: 22, 28, 26, 24.

```
1) Sprint Velocity = (22 + 28 + 26 + 24) ÷ 4 = 100 ÷ 4 = 25
```

```
2) Focus Factor    = 25 ÷ (6 × 8) = 25 ÷ 48 = 0.52
```

```
3) Working days    = 3 weeks × 5 = 15
```

```
   Capacity        = 0.52 × (6 × 8 × 15) = 0.52 × 720 ≈ 375 hours
```

الـ Focus Factor هنا `0.52`، أقل من المدى المثالي.

الفقرة 4: الـ backlog الجاي 30 نقطة، هل الخطة ممكنة؟ النسخة المحلولة اختارت **Prudent**، لأن 30 أكبر من متوسط الفريق 25. والقاعدة المكتوبة فيها:

المقارنة	الحكم
الـ Backlog قريب من الـ Velocity	**Feasible**
الـ Backlog أكبر بشوي	**Prudent**
الـ Backlog أكبر بكثير	**Not feasible**

اختيار النموذج (4 درجات في كل ميد)

إذا شفت في وصف المشروع	اختر
عميل متاح يومياً، متطلبات تتغير كثير، فريق صغير في مكان واحد، وقت قصير.	**Agile**
متطلبات ثابتة، فريق كبير أو موزّع، وثائق وتواقيع، أنظمة وقوانين.	**Plan-Driven**
كلمات مثل COTS أو vendor SDKs أو integrate، يعني مكونات جاهزة.	**Component-Based**

مشاريع الميد الأول وحلها:

المشروع	الجواب	السبب
Smart-City Sensor Hub	**Component-Based**	لازم يتكامل مع COTS gateways و vendor SDKs.
Banking Regulatory Reporting	**Plan-Driven**	متطلبات ثابتة بالأنظمة، 30 شخص، وثائق وتواقيع.
Campus Social-Learning MVP	**Agile**	ملاحظات يومية، 6 مطورين في مكان واحد، إطلاق خلال أسابيع.
Medical Device Companion App	**Plan-Driven**	معايير طبية ووثائق life-cycle إلزامية.

ميدين عيّنة صغيرة. تعطيك شكل الأسئلة، وليست ضمان للميد الجاي.

10 أخطاء شائعة

الخطأ	الصح
تقسم الـ Velocity على عدد الأشخاص بس	تقسم على (الأشخاص × ساعات اليوم).
تحسب الأسبوع 7 أيام	الأسبوع 5 أيام شغل.
تنسى تضرب في الـ Focus Factor	الـ Capacity = FF × (people × hours × days).
تخلط Refactoring مع Pair programming	**Refactoring** تحسين الكود. **Pair programming** مبرمجين مع بعض.
تختار Agile لمشروع فيه قوانين ووثائق	هذا **Plan-Driven**.

11 اختبر نفسك

1. فريق من 4، يوم العمل 8 ساعات، آخر 5 دورات: 18، 22، 20، 24، 21. احسب الـ Velocity والـ Focus Factor.
2. لنفس الفريق، احسب الـ Capacity لدورة مدتها أسبوعين.
3. أي ممارسة XP تعني دمج كل مهمة مكتملة بعد نجاح الاختبارات؟
4. مشروع: 5 مطورين في مكتب واحد، العميل يعطي ملاحظات يومياً، والمزايا تتغير كل أسبوع. أي نموذج؟
5. وش الأجزاء الثلاثة لـ User story؟

الحلول

```
1) Velocity = (18 + 22 + 20 + 24 + 21) ÷ 5 = 105 ÷ 5 = 21
```

```
   Focus Factor = 21 ÷ (4 × 8) = 21 ÷ 32 = 0.656
```

```
2) Working days = 2 × 5 = 10
```

```
   Capacity = 0.656 × (4 × 8 × 10) = 0.656 × 320 = 210 hours
```

#	الجواب
3	**Continuous integration**.
4	**Agile**: فريق صغير، عميل حاضر، ومتطلبات متغيرة.
5	**Who** و **What** و **Why**.

أسئلة اختبر نفسك من إعدادي، وليست من ميدات سابقة.